Autonomous Agents for Battleship: A Comparison of Probability-Based and Reinforcement Learning Approaches

BY

VENKATASHIVASAI MUPPIDI

FINAL PROJECT

Submitted in partial fulfillment of the requirements for Foundations of Artificial Intelligence

Northeastern University

Spring 2026

TABLE OF CONTENTS

Content	Page No.
1. Introduction	1
2. Methods	2
2.1 Game Environment	2
2.2 Probability-Based Agent	3
2.3 Reinforcement Learning Agent	5
2.4 Evaluation Methodology	6
3. Results	7
3.1 Performance Comparison	7
3.2 Distribution Analysis	8
3.3 Statistical Significance	10
4. Conclusions	11
5. Works Cited	13

1 Introduction

Battleship is a classic two-player strategy game where each player attempts to locate and sink their opponent's fleet of ships hidden on a grid. The game presents an interesting AI challenge because it is a partially observable environment: players cannot see enemy ship locations and must infer positions based solely on attack outcomes. This makes Battleship an ideal testbed for studying how AI agents reason under uncertainty.

The standard version uses a 10×10 grid with five ships: Carrier (5 cells), Battleship (4 cells), Cruiser (3 cells), Submarine (3 cells), and Destroyer (2 cells). Ships may be placed horizontally or vertically but cannot overlap. Players take turns calling coordinates to attack, receiving "hit" or "miss" feedback, until one player sinks all enemy ships.

From an AI perspective, Battleship presents several challenges. First, partial observability means the agent cannot directly observe the environment state and must maintain beliefs about ship locations. Second, the game requires balancing exploration (searching for ships) and exploitation (destroying found ships). Third, the state space contains approximately 10^{17} possible ship configurations, making exhaustive search infeasible.

The challenge of partial observability in reinforcement learning has received significant research attention. Zhang et al. (2024) address this in their work on provable representation with efficient planning for partially observable reinforcement learning [1]. Their research demonstrates that learning good state representations is crucial for effective decision-making when the agent cannot observe the complete environment state. This insight is directly relevant to Battleship, where an agent must represent game state in a way that captures uncertainty about hidden ship locations.

This project compares two fundamentally different approaches to building an autonomous Battleship agent. The first is a probability-based agent that uses domain knowledge and mathematical reasoning to calculate ship presence likelihood at each cell. This agent employs heuristics such as hunt/target mode switching, parity optimization, and edge penalties derived from human understanding of optimal strategy. The second is a reinforcement learn-

ing agent that learns through trial and error without explicit strategy knowledge. This agent uses Q-learning, the foundational value-based algorithm introduced by Watkins and Dayan (1992) [2], which enables learning optimal action-selection policies by iteratively updating state-action value estimates based on observed rewards.

The central research question is: How does a probability-based agent leveraging domain knowledge compare to an RL agent that must discover strategy through experience? This comparison illuminates tradeoffs between expert systems encoding human knowledge and learning systems acquiring knowledge autonomously—a decision arising in many AI applications. Understanding when domain knowledge outperforms learned strategies has practical implications for AI system design in domains ranging from game-playing to logistics and resource allocation.

This paper makes several contributions: (1) design and implementation of a complete Battleship game environment, (2) a sophisticated probability-based agent with multiple heuristics, (3) a Q-learning agent that learns from scratch, (4) rigorous experimental comparison across 1,000 games with statistical significance testing, and (5) discussion of implications for domain knowledge versus learning in game-playing AI.

2 Methods

2.1 Game Environment

The Battleship game environment was implemented in Python using Pygame for visualization [4]. The environment consists of a 10×10 grid where each cell can be empty, ship, hit, miss, or sunk. The standard fleet means 17 cells contain ships, yielding a 17% base hit rate for random shooting. Ships can be placed horizontally or vertically but cannot overlap or extend beyond boundaries.

The environment supports both human play through a graphical interface and automated agent evaluation at high speed. The interface displays the player's grid showing ship po-

sitions, the enemy grid showing attack results, ship status panels, and real-time statistics including shots, hits, misses, accuracy, and ships sunk. Figure 1 shows the game interface during play.

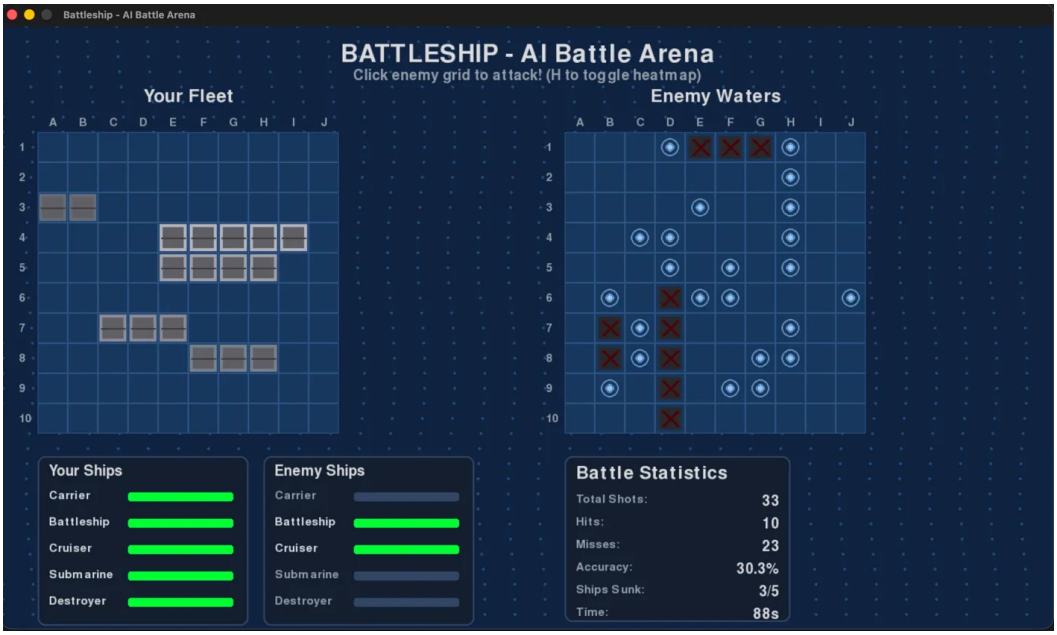

Figure 1: The Battleship game interface showing a game in progress. The left grid displays the player's fleet, the right grid shows attack results on enemy waters (red X for hits, blue circles for misses), and the bottom panels display ship status and battle statistics.

For automated evaluation, the environment runs games without visualization at high speed. Ship configurations are generated randomly each game, ensuring agents are evaluated across diverse scenarios. The environment provides feedback after each attack indicating hit/miss status and whether a ship was sunk.

2.2 Probability-Based Agent

The probability-based agent uses mathematical reasoning and domain-specific heuristics for targeting decisions. The agent maintains a probability map estimating ship presence like-

lihood at each unexplored cell, updated after each shot based on outcomes and remaining ship constraints.

The agent operates in two distinct modes. In hunt mode, the agent searches for ships by selecting targets based on probability scores. The calculation considers all possible placements of remaining ships and counts how many valid placements cover each cell—cells coverable by more placements receive higher scores. In target mode, activated after detecting a hit, the agent focuses on destroying the located ship by systematically attacking adjacent cells.

Figure 2 demonstrates the probability heatmap visualization. Cells are color-coded from red (high probability) to blue (low probability), showing how the agent concentrates search on regions that could accommodate remaining ships while avoiding areas near miss clusters.

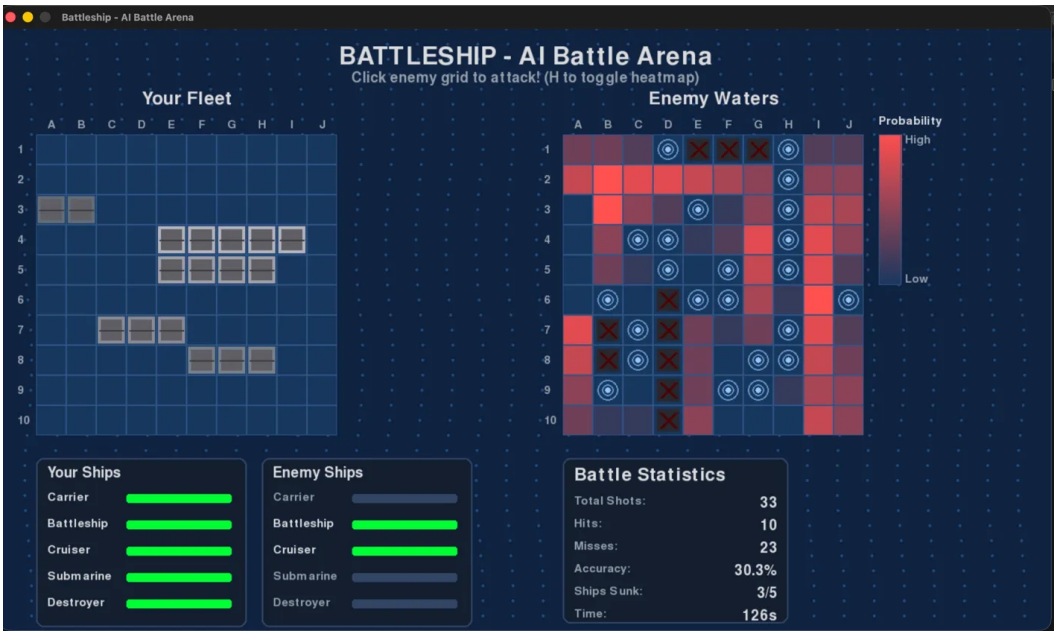

Figure 2: The probability heatmap overlay visualizing ship presence likelihood. Red indicates high probability and blue indicates low probability.

Several heuristics enhance performance. Parity optimization exploits the fact that the

smallest ship (Destroyer) occupies 2 cells, meaning an optimal search pattern resembles a checkerboard—by preferring cells where row and column indices sum to an even number, the agent can find all ships while potentially skipping half the cells. Edge penalty reduces probability near board edges because ships have fewer valid placement options near boundaries. Density bonus increases probability for cells surrounded by unexplored areas. Miss cluster penalty reduces probability near multiple misses, as clustered misses suggest that area lacks ships.

In target mode, after a single hit the agent attacks four adjacent cells in probability order. After two consecutive hits, it determines ship orientation (horizontal/vertical) and attacks only along that line until the ship sinks, then returns to hunt mode.

2.3 Reinforcement Learning Agent

The reinforcement learning agent uses Q-learning (Watkins & Dayan, 1992) [2] to learn Battleship strategy through trial and error. Q-learning is a model-free algorithm that learns action values by maintaining a table of Q-values representing expected cumulative reward for taking an action in a state and following the optimal policy thereafter. The algorithm updates these values using the Bellman equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)] \tag{1}$$

where α is the learning rate, γ is the discount factor, r is the reward, and s' is the next state. Through repeated experience, Q-values gradually converge to optimal values.

The initial approach attempted Deep Q-Networks (DQN), the architecture introduced by Mnih et al. (2015) [3] that achieved human-level performance on Atari 2600 games. DQN combines Q-learning with deep neural networks, using experience replay and target networks to stabilize training. This architecture was chosen for its demonstrated success in game-playing and ability to learn spatial patterns through convolutional layers.

However, DQN failed for Battleship. Despite thousands of training episodes on both CPU and GPU, the network's loss became unstable and performance remained at random-play level (approximately 95 shots per game, 18% accuracy). This failure highlights an important distinction: while Atari games provide rich visual feedback where pixels encode game state, Battleship's partial observability means the board representation lacks information about hidden ship locations. This challenge aligns with Zhang et al. (2024) [1], who demonstrate that effective representation learning is crucial in partially observable environments.

Given DQN's failure, the agent was redesigned using tabular Q-learning with hand-crafted state features. The state representation encodes current mode (hunt/target), hit and miss counts, and target hit configuration when in target mode. This compact representation reduces state space to manageable size while capturing essential decision-making information.

The action space consists of all 100 cells, with invalid actions (previously attacked cells) masked during selection. The agent uses epsilon-greedy action selection, starting with high exploration ($\epsilon = 1.0$) and gradually reducing to low exploration ($\epsilon = 0.05$).

The reward structure guides learning: misses receive -0.5 (discouraging random shooting), hits receive +2.0, sinking ships provides +5.0, and winning provides +20.0. The agent also incorporates basic heuristics for initial guidance: position values prefer center cells, and hunt/target mode logic switches after hits. These provide a foundation that Q-learning refines through experience.

Training used 5,000 episodes with learning rate 0.15 and discount factor 0.95, completing in approximately 32 seconds—demonstrating tabular Q-learning's computational efficiency compared to deep learning.

2.4 Evaluation Methodology

Both agents were evaluated through systematic experimental comparison. Each agent played 1,000 games against randomly generated ship configurations. To ensure fair comparison, both agents played against identical configurations, eliminating variance from opponent difficulty

differences.

The primary metric is shots required to win, directly measuring efficiency (lower is better). Secondary metrics include accuracy (percentage of shots hitting ships) and shot count distribution.

Statistical analysis determined whether differences are significant. The independent samples t-test compared mean shot counts. The Mann-Whitney U test provided a non-parametric alternative not assuming normal distributions. Cohen's d measured effect size, indicating practical significance beyond statistical significance [5].

Experiments used Python 3.14 on a MacBook Air with Pygame 2.6.1 [4], SciPy 1.12 [6], NumPy [7], and Matplotlib 3.8 [8].

3 Results

3.1 Performance Comparison

The experimental comparison revealed substantial performance differences between agents. Table 1 presents summary statistics across 1,000 games.

Table 1: Performance Comparison (1,000 games per agent)

Metric	Probability Agent	RL Agent
Average Shots	49.75	61.90
Standard Deviation	13.00	14.39
Median Shots	48.0	61.0
Minimum Shots	26	28
Maximum Shots	103	100
Average Accuracy	36.17%	29.08%

The probability agent required 49.75 shots on average versus 61.90 for the RL agent—a

12.15 shot difference representing 24% improvement. The probability agent also achieved higher accuracy (36.17% vs 29.08%). Figure 3 shows average shots comparison with standard deviation error bars. The probability agent's advantage is clear, with non-overlapping error bars indicating statistically meaningful difference.

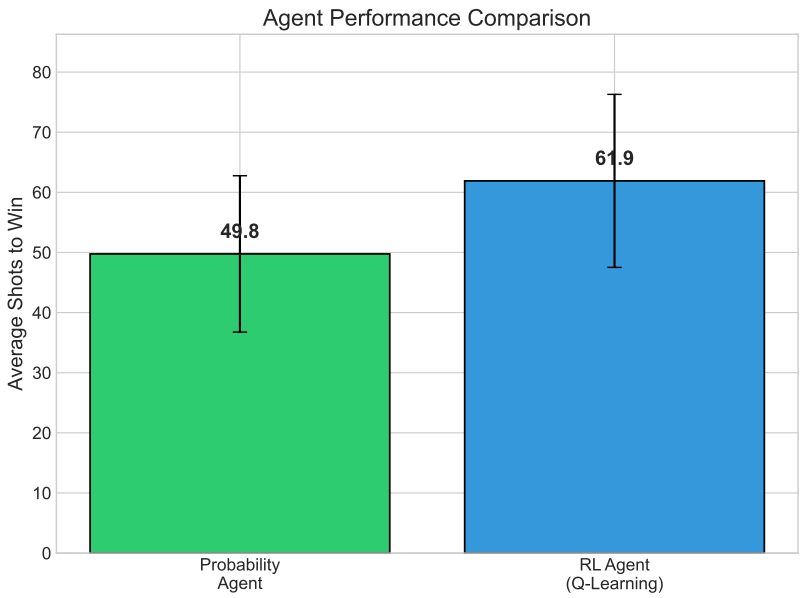

Figure 3: Average shots to win for both agents with standard deviation error bars.

3.2 Distribution Analysis

Figure 4 presents shot distribution histograms. The probability agent's distribution (green) peaks around 45-50 shots, while the RL agent (blue) peaks around 55-65 shots. Dashed lines mark means (49.8 and 61.9). Substantial overlap indicates both agents experience good and poor games, but the probability agent achieves good performance more frequently.

Figure 5 shows violin plots comparing full distributions. The probability agent's distribution concentrates in the 40-55 shot range with lower median, while the RL agent centers higher (50-75 range). The probability agent shows a longer tail toward 100 shots for difficult configurations but overall clusters more tightly around its median. The tighter clustering of the probability agent's distribution suggests more consistent performance regardless of ship

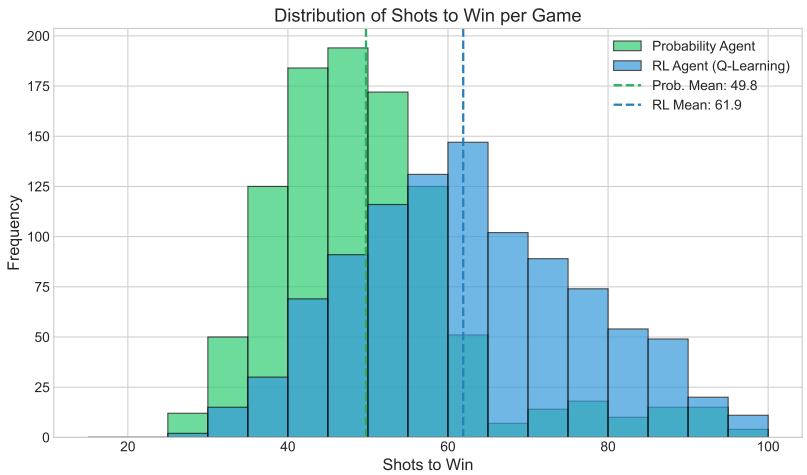

Figure 4: Distribution of shots to win per game. Dashed lines indicate mean values.

configuration.

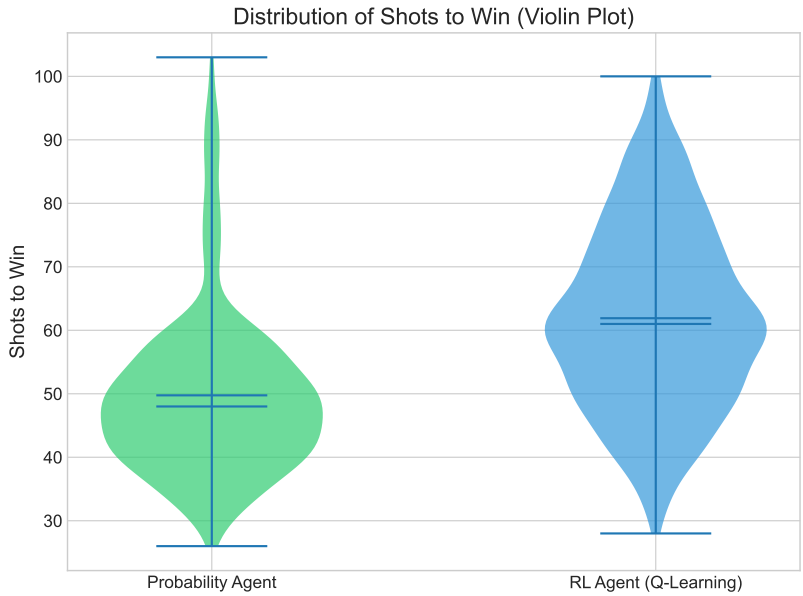

Figure 5: Violin plot showing the full distribution of shots to win for both agents.

Figure 6 compares accuracy. With 17 cells containing ships, random shooting yields 17% accuracy. Both agents substantially exceed this baseline, but the probability agent's 7.1 percentage point advantage directly translates to fewer wasted shots.

Figure 7 shows cumulative distribution curves—the percentage of games won within each shot count. The probability agent's curve rises more steeply: it wins 50% of games within

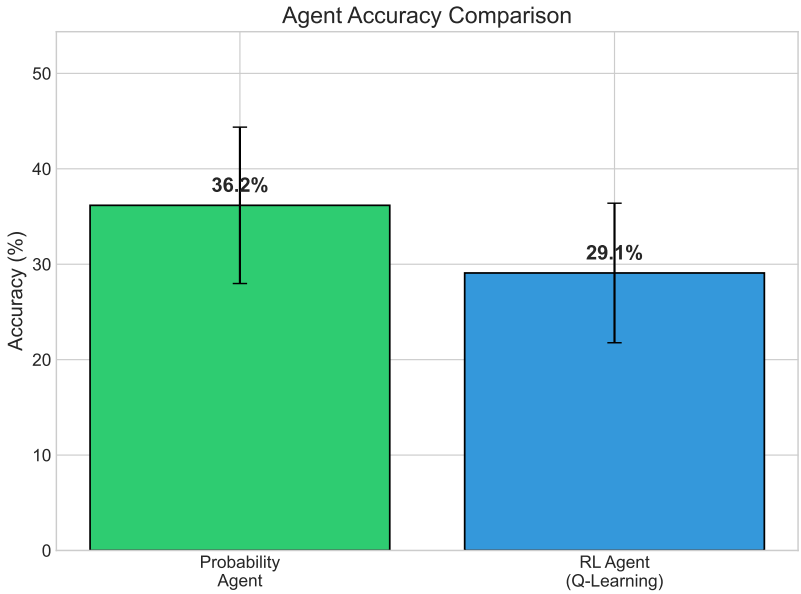

Figure 6: Accuracy comparison showing the percentage of shots that hit ships.

approximately 48 shots versus 61 for RL, and 90% within approximately 65 shots versus 80 shots. This visualization demonstrates the practical performance gap across all games.

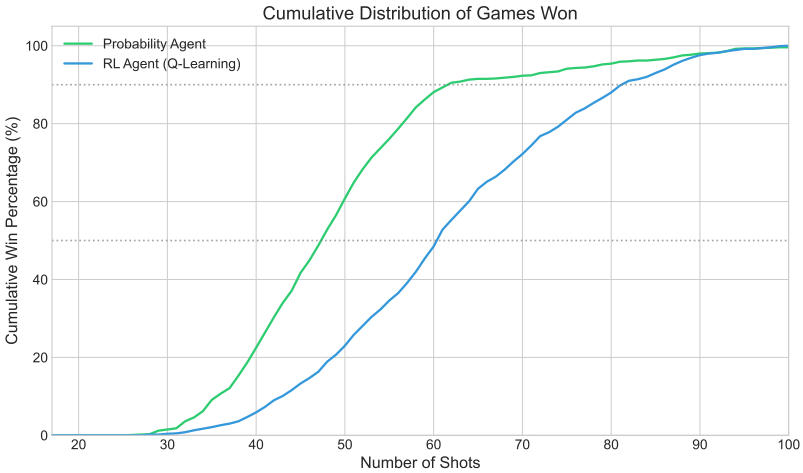

Figure 7: Cumulative distribution of games won by number of shots.

3.3 Statistical Significance

Statistical analysis confirmed highly significant differences. The t-test yielded $p = 7.68 \times 10^{-80}$, far below any conventional threshold. The Mann-Whitney U test confirmed this

with $p = 1.78 \times 10^{-87}$. Cohen's $d = 0.886$ indicates a large effect size. According to Cohen (1988) [5], effect sizes of 0.2 are small, 0.5 are medium, and above 0.8 are large. The observed 0.886 represents not just statistical significance but large practical difference between agents. The extremely low p-values mean that if both agents were actually performing equally well, the probability of observing a difference this large by chance would be essentially zero. In practical terms, this confirms that the probability agent's superior performance is a genuine result of its design rather than random variation across games.

4 Conclusions

This project compared two approaches to autonomous Battleship agents: a probability-based agent encoding domain knowledge and a reinforcement learning agent discovering strategy through experience. The probability agent significantly outperformed the RL agent, requiring approximately 12 fewer shots per game. This difference is both statistically significant ($p < 0.001$) and practically meaningful (Cohen's $d = 0.886$).

The probability agent's superior performance stems from effective domain knowledge use. It explicitly encodes heuristics derived from human understanding: probability-based targeting, parity optimization, hunt/target switching, and edge penalties. These allow immediate effective decisions without training—essentially embodying decades of human Battleship experience.

The RL agent learned reasonable but inferior behaviors. It developed hunt/target modes, learned to prefer center cells, and followed ship orientations after hits. However, it failed to discover sophisticated strategies like optimal parity patterns or precise probability calculations. This reflects reinforcement learning's fundamental challenge: discovering complex strategies from scalar reward signals is difficult, especially in partially observable environments.

The DQN failure is particularly noteworthy. Mnih et al. (2015) [3] demonstrated remark-

able DQN success on Atari, where screen pixels directly encode relevant game state. Battleship differs fundamentally: its partial observability means board representations lack information about hidden ships. Zhang et al. (2024) [1] address precisely this challenge, showing that effective state representations are critical in partially observable settings. The successful tabular Q-learning used hand-crafted features capturing relevant information (mode, hit locations, orientations), confirming that representation design remains crucial even with learning algorithms.

This project has limitations. The RL agent trained for only 5,000 episodes; additional training might improve performance, though diminishing returns were apparent. The comparison involves single designs of each approach; variants might perform differently. Both agents played against random ship placements; strategic opponents placing ships unexpectedly might change results.

Future work could explore hybrid approaches combining probability calculations with learned refinements. More sophisticated partial observability algorithms, such as those proposed by Zhang et al. (2024) [1], might overcome standard Q-learning's limitations. Extending comparison to other partially observable games would test whether findings generalize.

In conclusion, domain knowledge provides substantial advantages in structured domains like Battleship. While reinforcement learning offers appealing learning-without-programming properties, the probability agent's superior performance shows that expert knowledge, when available, remains valuable. These findings suggest that in domains with well-understood structure, encoding domain knowledge may be more effective than expecting agents to rediscover it through learning. The most promising direction may combine both worlds: using learning to refine and extend domain knowledge rather than replace it entirely.

5 Works Cited

[1] H. Zhang, T. Ren, C. Xiao, D. Schuurmans, and B. Dai, "Provable representation with efficient planning for partially observable reinforcement learning," *Proceedings of the 41st International Conference on Machine Learning (ICML)*, vol. 235, pp. 59759–59782, 2024.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3-4, pp. 279–292, 1992.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[4] Pygame. "Pygame documentation." https://www.pygame.org/docs/

[5] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Lawrence Erlbaum Associates, 1988.

[6] The SciPy Community. "SciPy documentation." https://docs.scipy.org/

[7] NumPy Developers. "NumPy documentation." https://numpy.org/doc/

[8] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.